



Leibniz  
Universität  
Hannover

# Stochastic Finite Element Methods

## #4: Sampling Techniques

Prof. Dr.-Ing. Udo Nackenhorst  
Dr. Zhibao Zheng  
IBNM – Leibniz Universität Hannover

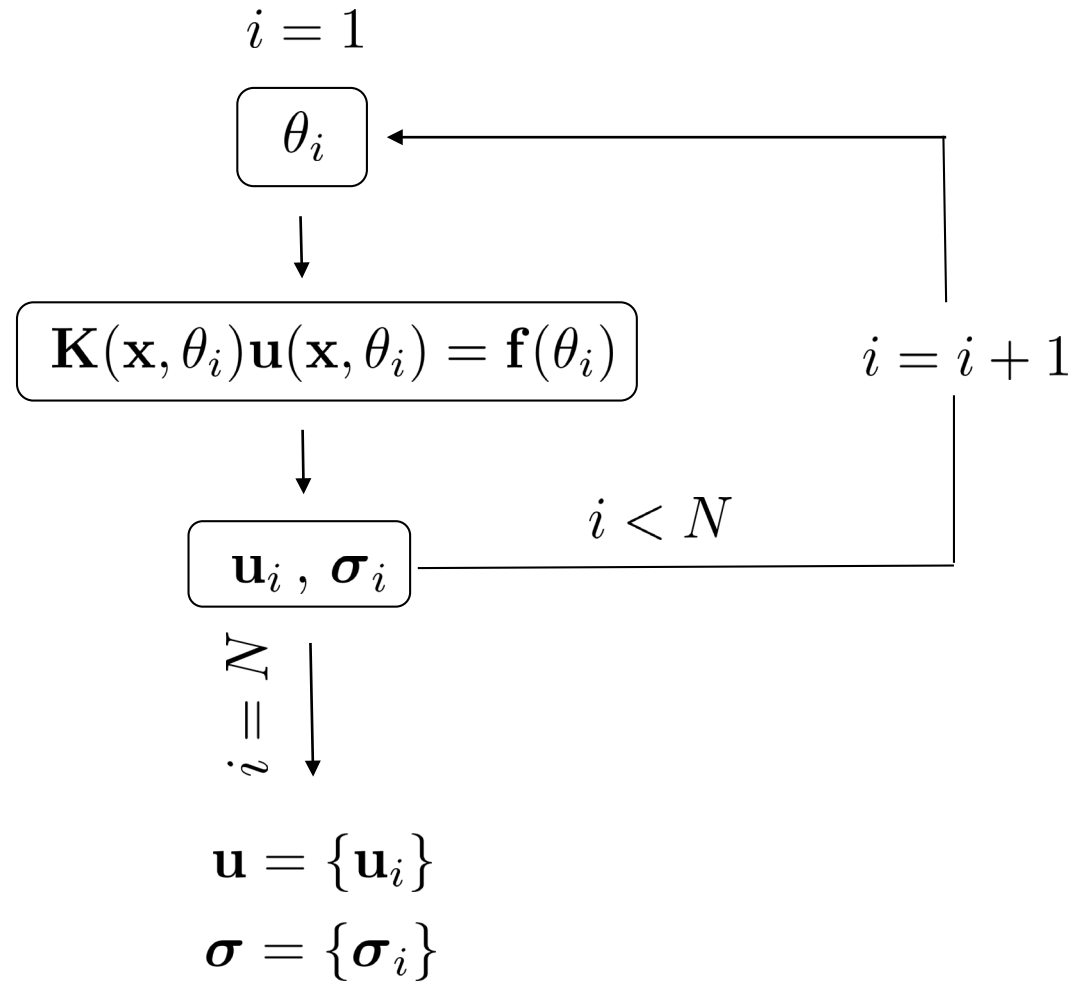
---

# Introduction

random number  
generator

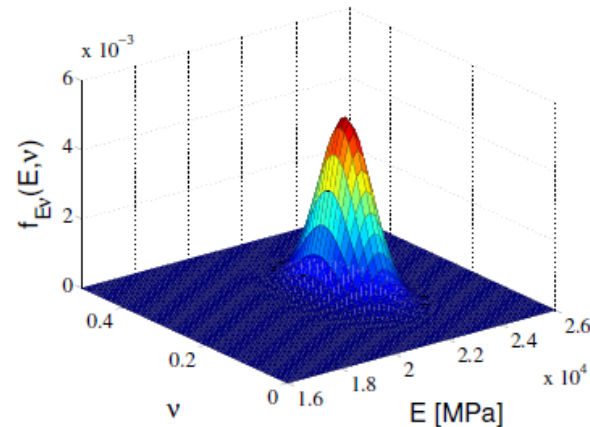
deterministic  
solver

stochastic output



# Sampling a continuous RV

Discrete sets of data to approximate the random variable



$$\begin{array}{ll} E_1 = 199 \text{ GPa} & \nu_1 = 0.27 \\ E_2 = 210 \text{ GPa} & \nu_2 = 0.31 \\ E_3 = 195 \text{ GPa} & \nu_3 = 0.30 \\ E_4 = 203 \text{ GPa} & \nu_4 = 0.28 \\ \vdots & \vdots \\ E_{1000} = 196 \text{ GPa} & \nu_{1000} = 0.32 \end{array}$$

Joint probability density  
function  $f_{E\nu}$



Generating discrete sets  
of random data

$$\{E_i, \nu_i\}_N$$

= samples

Random Number Generator: Numerical tool for generating discrete sets. Range of realizations large enough → resulting scatter is of little consequence in modeling a continuous distribution.

# Outline

- Sampling techniques to provide a set of random variables describing the uncertain information:
  - Monte Carlo Sampling
  - Quasi Monte Carlo Sampling
  - Latin Hypercube Sampling
- Effort and accuracy of the different methods

Lit: Gentle J. E.: Random Number Generation and Monte Carlo Methods,  
Springer, 2003

# Monte Carlo Sampling (MCS)

$$\mu_X = \int_{-\infty}^{\infty} x f_X(x) \, dx \approx \frac{1}{n} \sum_{i=1}^n x(\xi_i) f_X(\xi_i)$$

- Simple generator and may be the most widespread sampling technique
- Independent realisations of random inputs with respect to their probability distribution using pseudo-random generators
- Provide a set of random numbers by using deterministic rules
- Independent of the dimension of the probability space
- Disadvantage: poor convergence  $\mathcal{O}\left(\frac{1}{\sqrt{n}}\right)$

# Linear Congruential Generator

A uniform pseudo-random generator

LCG generates pseudo-random numbers with uniform distribution on  $]0, 1[$

$$u_i^{\text{LCG}} = \frac{z_i}{m} \quad \text{with} \quad z_i = (az_{i-1} + c) \bmod(m), \quad \forall i \geq 0$$

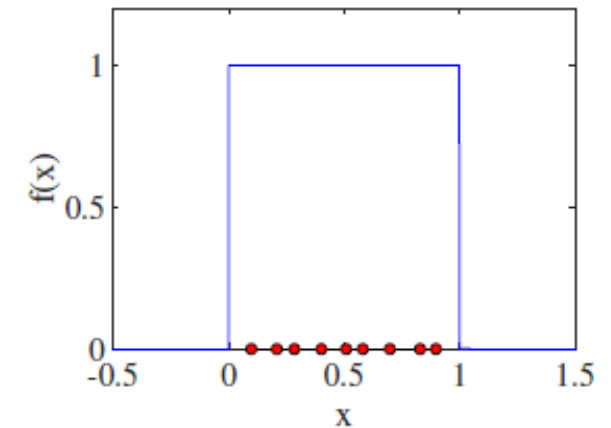
with

- multiplier  $a$
- increment  $c$
- modulus of entire numbers  $m$

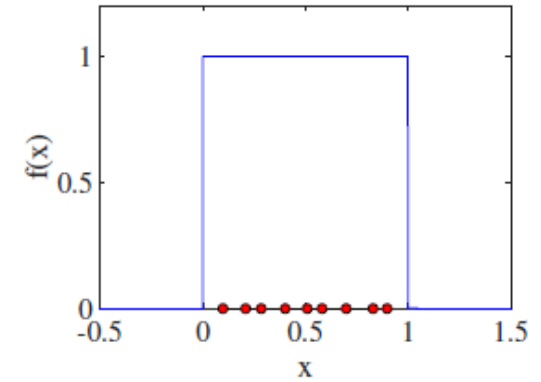
$\bmod$  reminder in Euclidian division

$z_0$  is the seed of the sequence  $\longrightarrow$  repeatability of a set  $\{u_i\}_N$

Frequently used data:  $a = 7^2$ ,  $c = 0$ ,  $m = 2^{31} - 1$



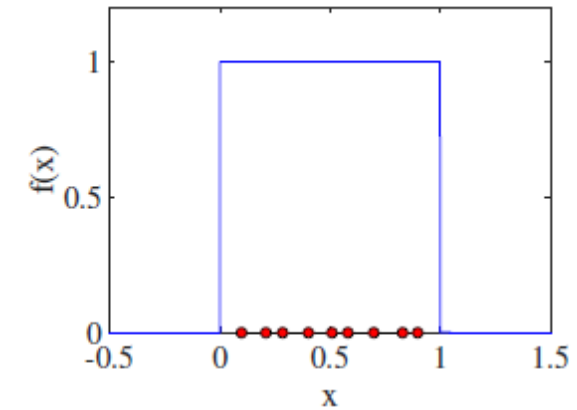
# Linear Congruential Generator



## Properties:

- the sequence  $\{u_i\}_N^{\text{LCG}}$  is periodic with the period  $m$ , which might be too small
- usage limited to  $N < m$  random numbers
- robustness questionable

# Mersenne Twister Generator



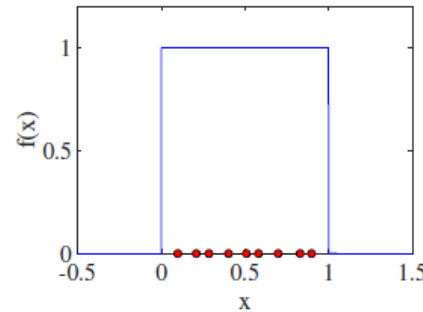
- Widespread pseudo-random number generator
- Based on operations on bytes rather than classical arithmetic operations
- Provides a large period ( $2^{19937} - 1$ )
- Default implementation in most toolboxes, e.g. Matlab, Python, for classical probability distributions
- Repeatability at restart

# Shuffling

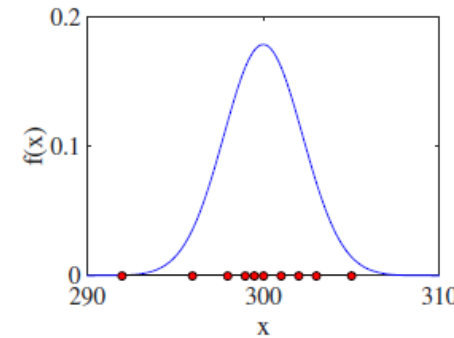
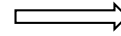
The output-stream of a pseudo-random generator is shuffled by using another algorithm to permute sub-sequences:

- to increase the period
- to break up the lattice structure

# How to obtain a non-uniform sample?



Transformation



Discrete random set  $\{u_i\}_N$   
uniformly distributed on  $]0, 1[$

$\{x_i\}_N$  with PDF  $f_x$   
and CDF  $F_x$

- Inverse CDF method
- Acceptance / Rejection method
- Stationary distribution of Markow Chains
- Decomposition of distributions



refer to Risk and Reliability Classes

# Quasi Monte-Carlo methods

- Low-discrepancy deterministic sequences based on *van der Corput* sequences, e.g. *Hammersley*, *Halton*, *Sobol*, *Niederreiter*, *Faure*
- Discrepancy:
  - A sequence of real numbers is equidistributed on  $[a, b]$  when it satisfies for any subinterval  $[c, d] \subset [a, b]$

$$\lim_{N \rightarrow +\infty} \frac{|\{x_1, \dots, x_N\} \cap [c, d]|}{N} = \frac{d - c}{b - a}$$

- The discrepancy for a sequence

$$D_N = \sup_{a \leq c \leq d \leq b} \left| \frac{|\{x_1, \dots, x_N\} \cap [c, d]|}{N} - \frac{d - c}{b - a} \right|$$

- A sequence is equidistributed if the discrepancy  $D_N$  tends to zero when  $N$  tends to infinity

# E.g. Sobol-Sequences

## Sobol-Sequences

$$x_i = \frac{b_0}{2} + \frac{b_1}{2^2} + \cdots + \frac{b_m}{2^{m+1}}$$

Integers  $b_k \in [0, 1]$  constructed such that

$$(i - 1)_2 = b_m b_{m-1} \cdots b_1 b_0$$

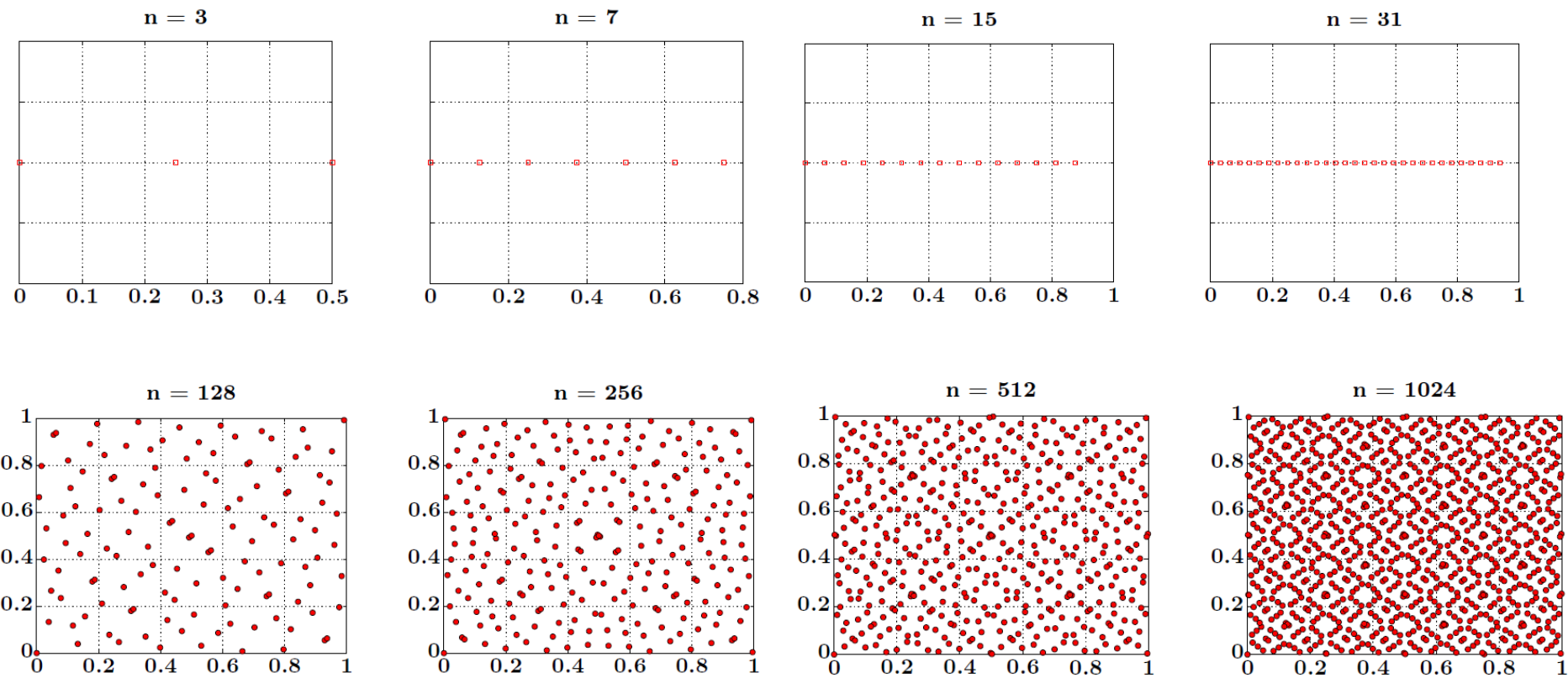
Theoretical (best) convergence rate  $\mathcal{O}(1/n)$

Matlab: *sobolset()*

Python: related libraries available

# E.g. Sobol-Sequences

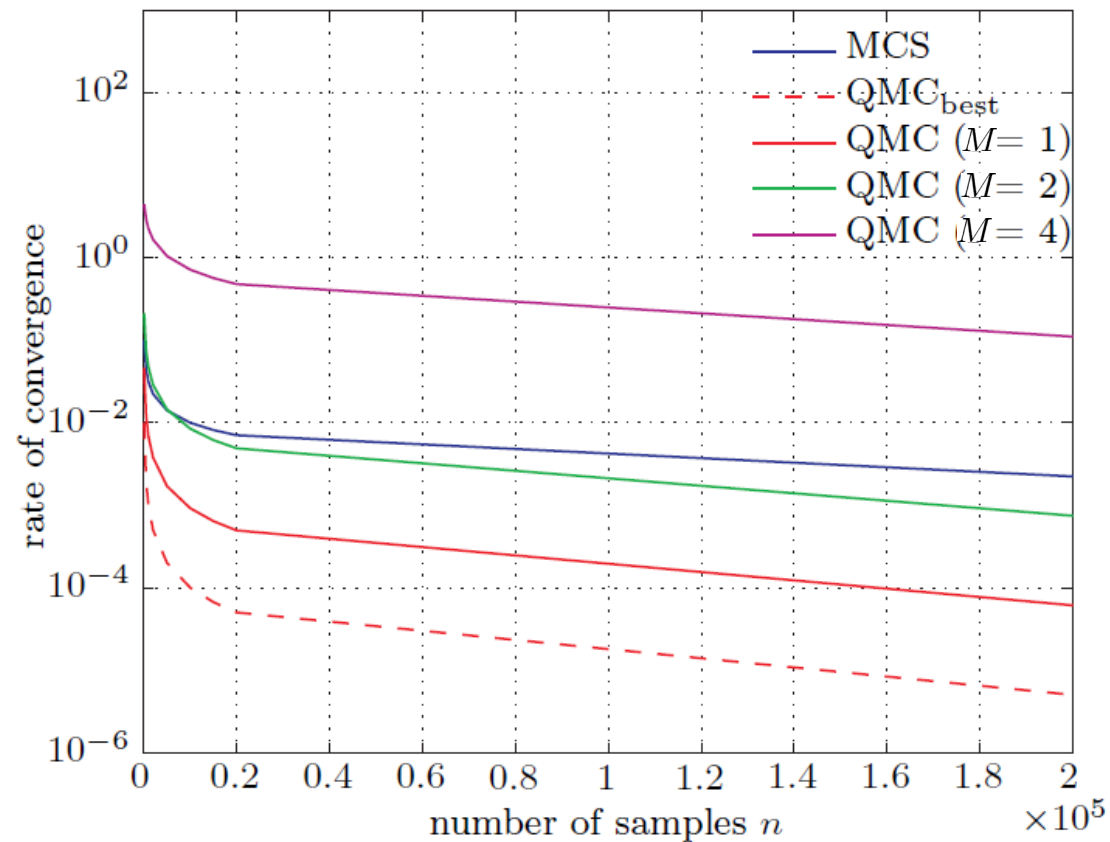
## Space filling with Sobol-Sequence



# Sobol Sequences

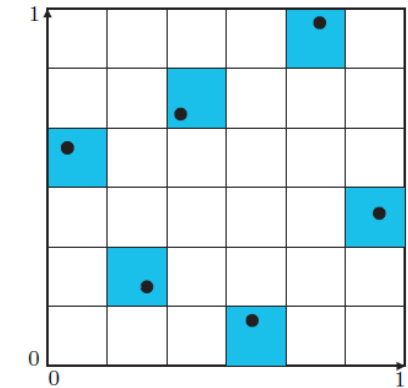
Convergence rate (upper bound)  $\mathcal{O}(\log^M(n)/n)$

$M$  : stochastic dimension



# Latin Hypercube Sampling (LHS)

- also called stratified sampling without replacement
- Good performance in multi-dimensional random spaces
- For statistical sampling, a Latin square is a  $N \times N$  array filled with  $N$  different symbols, each occurring once in each row and in each column.
- LHS is the generalization of this concept to an arbitrary number of dimensions



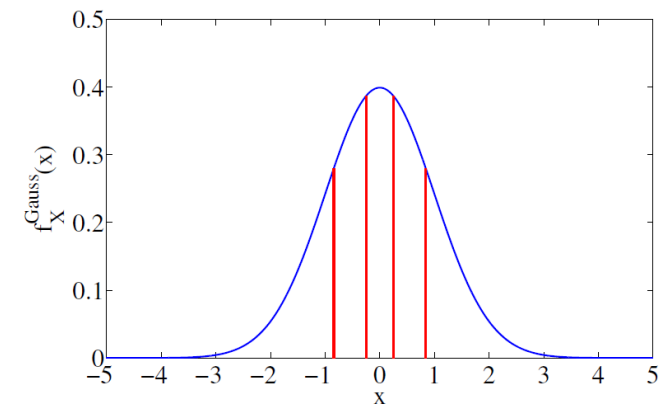
$$\{x^{\text{LHS}}\}_N \text{ with } \dim(X) = M$$

# Latin Hypercube Sampling (LHS)

3 steps for construction:

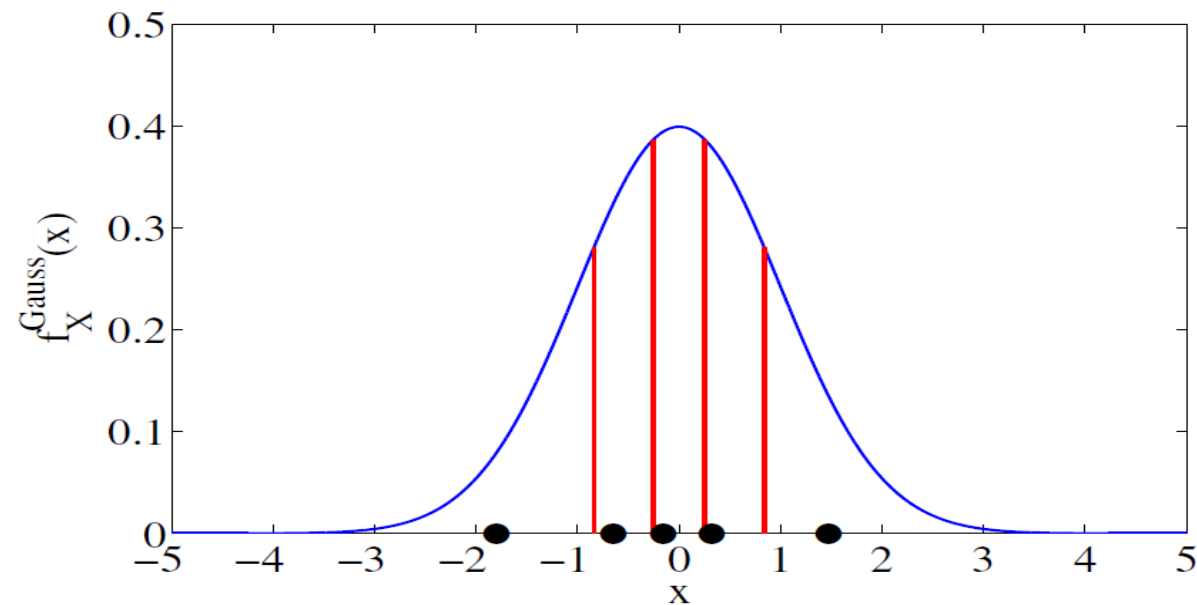
- 1st step: Independent stratification for each direction
  - Each dimension  $X_m$  is divided into  $N$  subintervals  $S_{mn}$  with the property, that each interval has the same probability  $p_{mn} = 1/N$
  - The bands of the stratification are wider when the probability density gets smaller
  - The same number of stratifications for all dimensions

Stratification of a normal distribution  
with five intervals



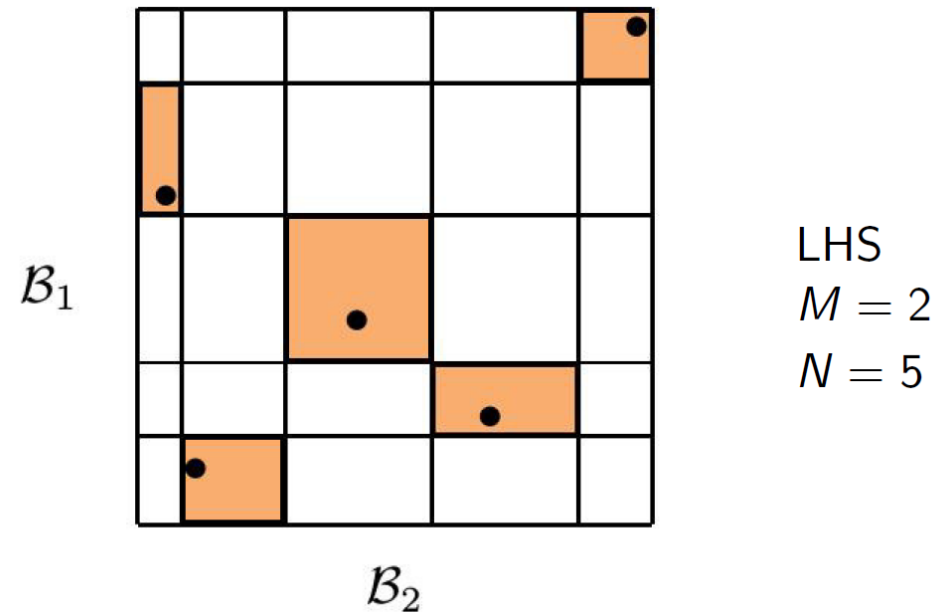
# Latin Hypercube Sampling

- 2nd step: Space one random value into each one-dimensional interval
  - For each dimension and for each interval, one value  $x_{mn}$  in the interval  $S_{mn}$  is chosen randomly



# Latin Hypercube Sampling (LHS)

- 3rd step: randomly pair the sampling of each dimension
  - The  $N$  values obtained for each dimension are randomly paired with the values sampled for the other dimensions



- This step requires an equal number of divisions for all dimensions

# Latin Hypercube Sampling (LHS)

- The number of combinations for LHS of  $N$  divisions and  $M$  dimensions is

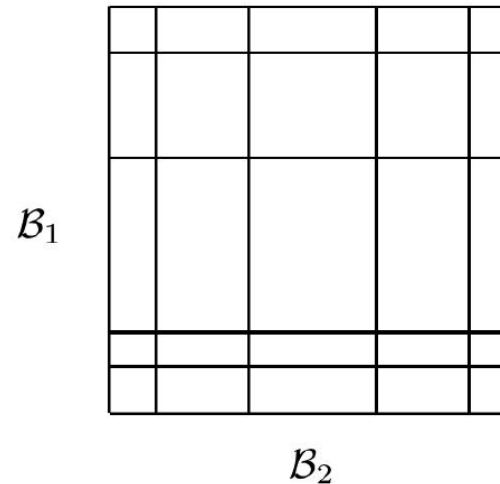
$$\left( \prod_{m=0}^{M-1} (N - m) \right)^{M-1} = (N!)^{M-1}$$

$N = 5, M = 2 \rightarrow 120$  possible combinations,

$N = 5, M = 3 \rightarrow 14400$  possible combinations.

# Orthogonal LHS

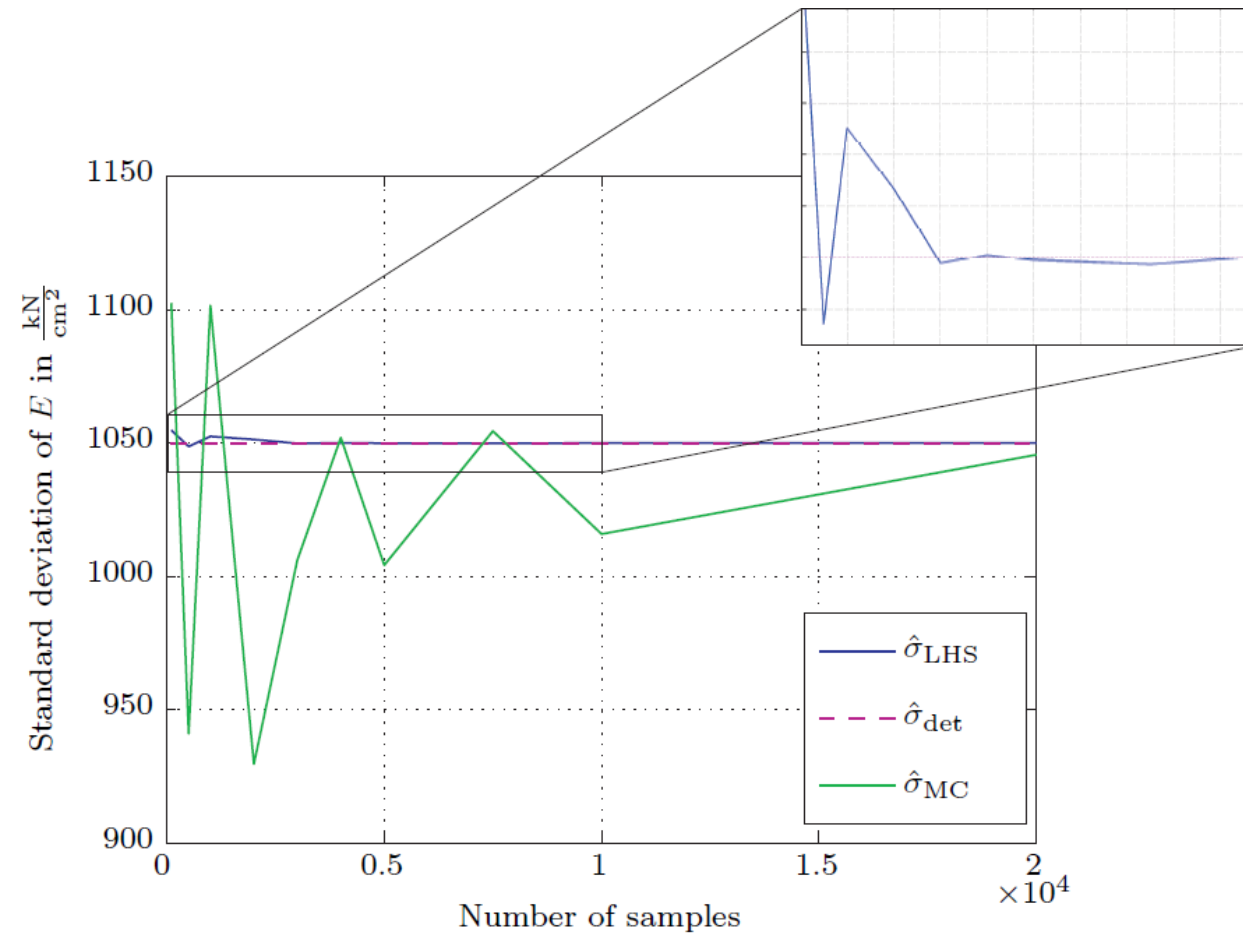
- The multi-dimensional sample space is subdivided into multi-dimensional intervals with the same probability



- For each dimension  $m$ :
  - A first random number is generated to determine which interval  $S_{mn}$  is selected,
  - A second random number is generated to determine the random value within each interval.

# Performance of LHS

e. g. Gaussian distributed Young's modules



# Judgment of the Accuracy

Requirements on the random number generator:

- Accurate representation of the distribution of a continuous RV by a discrete set of data
- Control over the repeatability and non-repeatability
- Portability
- Many mathematical criteria have been suggested to analyze the quality of RN generators, e.g.  $\chi^2$  – test etc.
- Optical visualization of low discrepancy distribution of random number generators
- In engineering typical: convergence of the variance

# Three Families of Sampling Methods

## MCS:

- One does not necessarily need to know the needed number of sample points in advance
- New sample points are generated based on knowledge of the previously generated sample points

## QMC:

- Deterministic sequence results in correlation between the points

## LHS:

- One must decide in advance how many sampling points are needed

# How to Reduce the Effort?

Mathematical Problem:

- SPDE  $\mathcal{L}_X(u) = \mathcal{F}$
- with random input parameters (known distribution)  $X$
- and stochastic output (QoI)  $Y$
- Goal: compute  $\mu(Y) = \mathbb{E}\{Y\}$  (or other statistical quantities)
- In practice we go for an approximate solution (FEM) by discretizing the SPDE in space  $\mathcal{L}_{h,X}(u_h) = \mathcal{F}_h$
- Approximate output  $Y_h = Y(u_h)$
- Monte Carlo Method  $Y_h^{(i)}, i = 1, \dots, N \rightarrow \mu_h^{\text{MC}} = \frac{1}{N} \sum_{i=1}^N Y_h^{(i)}$
- Mean squared error

$$\text{MSE}(\mu_h^{\text{MC}}) = \mathbb{E}\{(\mu(Y) - \mu_h^{\text{MC}})^2\} = \underbrace{(\mathbb{E}\{Y - Y_h\})^2}_{\text{discr.error}} + \underbrace{\frac{1}{N} \text{Var}(Y_h)}_{\text{MCerror}}$$

# Improvements on MC

Control Variate  $Z$

$$\mu_h^{\text{CV}} = \frac{1}{N} \sum_{i=1}^N (Y_j^{(i)} - \alpha Z^{(i)}) + \alpha \mathbb{E}\{Z\}$$

Where  $Z$  is a RV with known mean, correlated to  $Y_h$

$$\text{Var}(Y_{h,Z}) = \text{Var}(Y_h - \alpha Z) = \text{Var}(Y_h) + \alpha^2 \text{Var}(Z) - 2\alpha \text{Cov}(Y_h, Z)$$

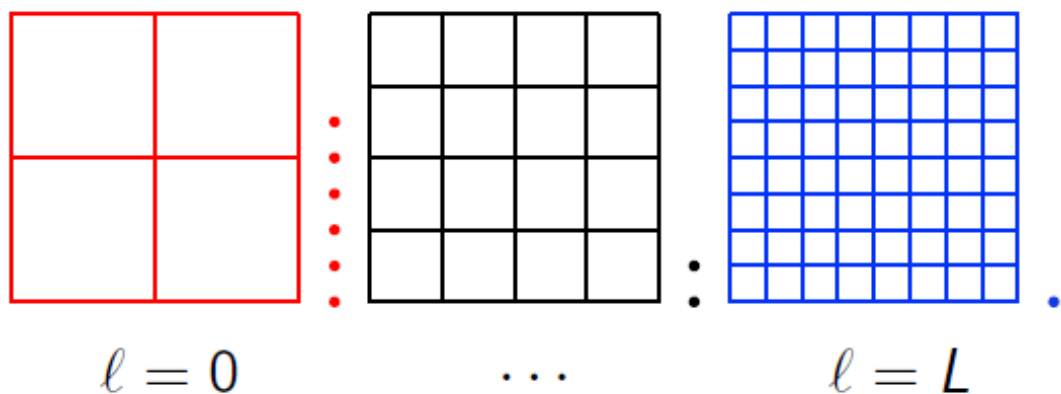
Variance reduction: for optimal  $\alpha$

$$\text{Var}(Y_{h,Z}) = \text{Var}(Y_h) \left( 1 - \frac{\text{Cov}(Y_h, Z)}{\text{Var}(Z)} \right) \leq \text{Var}(Y_h)$$

Choice for  $Z$

- Surrogate model  $Z = Y^{\text{surr}}$  with numerically optimized  $\alpha$   
     → *multi-fidelity Monte Carlo* [Peherstorfer et al. 2016]
- Coarser mesh, e.g.  $Z = Y_{2h}$  with  $\alpha = 1$   
     → *two-level Monte Carlo* [Heinrich 1998, Giles 2008]

# Multilevel Monte Carlo Method



- Sequence of refined meshes

$$h_0 > h_1 > \dots > h_L$$

- Sequence of sample sizes

$$N_0 > N_1 > \dots > N_L$$

[Nobile 2018]

- MLMC estimator

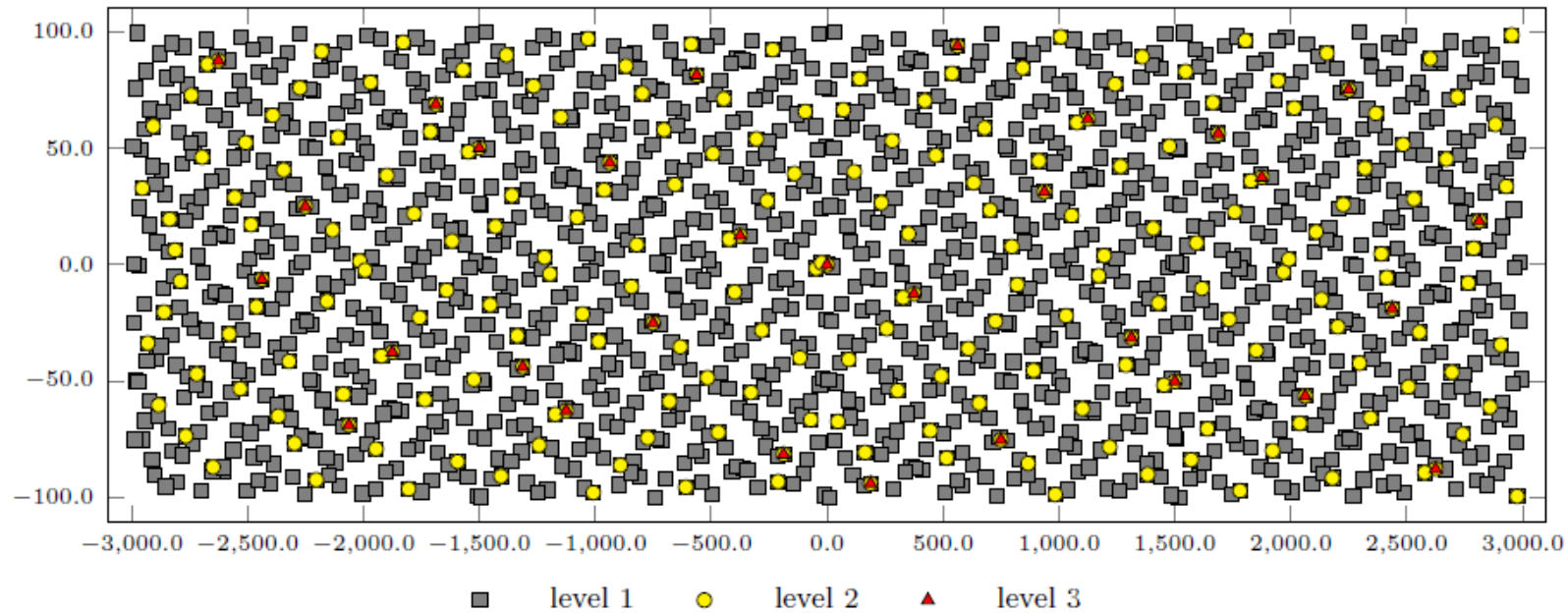
$$\begin{aligned} \mu_L^{\text{MLCM}} &= \frac{1}{N_0} \sum_{i=1}^{N_0} Y_0^{(i,0)} + \frac{1}{N_1} \sum_{i=1}^{N_1} (Y_1^{(i,1)} - Y_0^{(i,1)}) + \dots + \frac{1}{N_L} \sum_{i=1}^{N_L} (Y_L^{(i,L)} - Y_{L-1}^{(i,L)}) \\ &= \sum_{\ell=0}^L \frac{1}{N_\ell} \sum_{i=1}^{N_\ell} (Y_\ell^{(i,\ell)} - Y_{\ell-1}^{(i,\ell)}), \quad Y_{-1} = 0 \end{aligned}$$

- Mean squared error

$$\text{MSE}(\mu_h^{\text{MLMC}}) = \underbrace{(\mathbb{E}\{Y - Y_L\})^2}_{\text{discr. error level } L} + \underbrace{\sum_{\ell=0}^L \frac{1}{N_\ell} \text{Var}(Y_\ell - Y_{\ell-1})}_{\text{stat. error}}$$

# MLMC: Further Improvements

- Combination with nested Sobol sequences

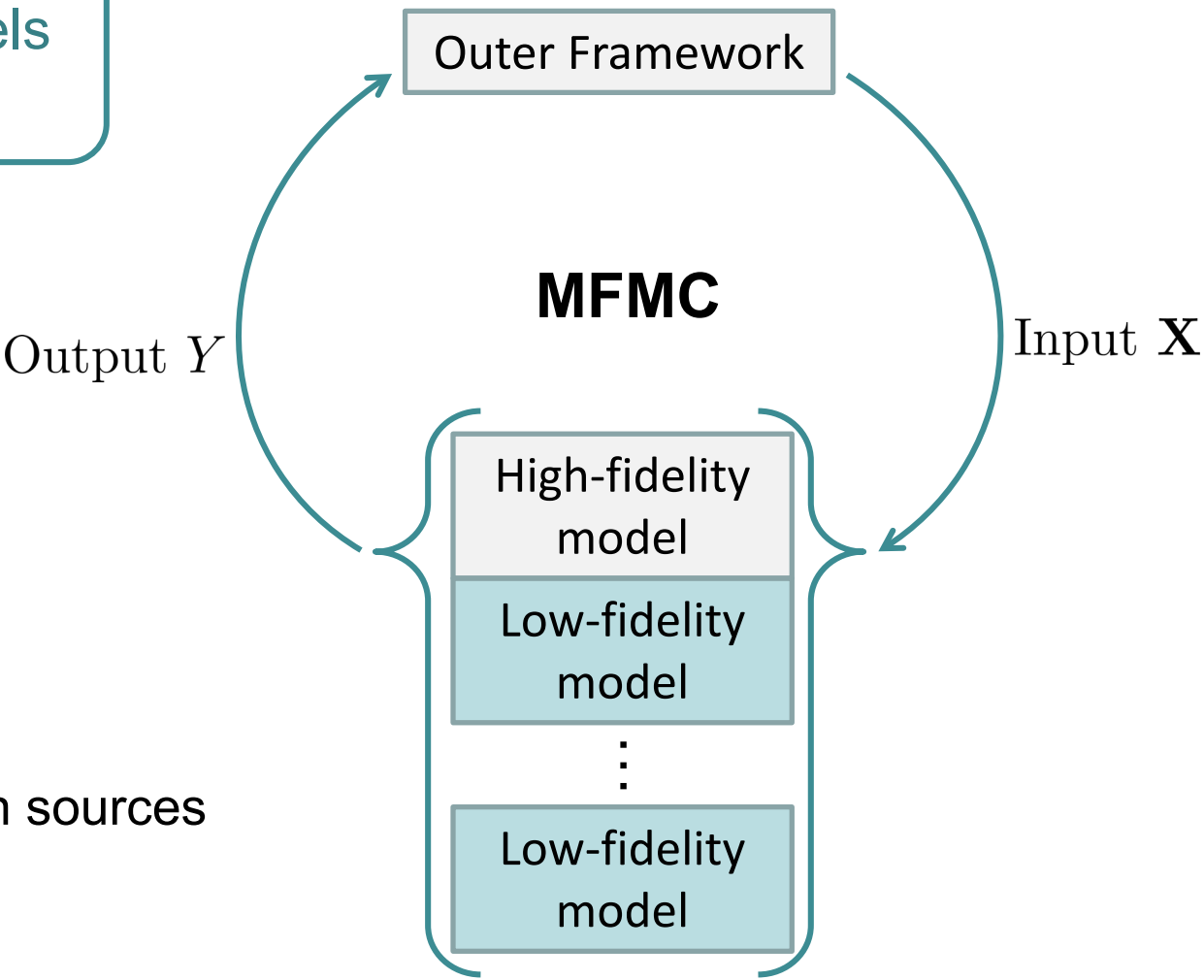


- Higher order moments etc.

# Multi-Fidelity Monte-Carlo

## Main idea

Combine **different models** to speed up MC



➔ different information sources

# Possible models

## High-fidelity model $f_1$

- ↑ High accuracy
- ↑ High calculation time
- ↓ Few calculations

### Examples:

- accurate FEM calculation

„truth“

approximation

## Low-fidelity models $f_2, f_3, \dots$

- ↓ Lower accuracy
- ↓ Low calculation time
- ↑ Many calculations

### Examples:

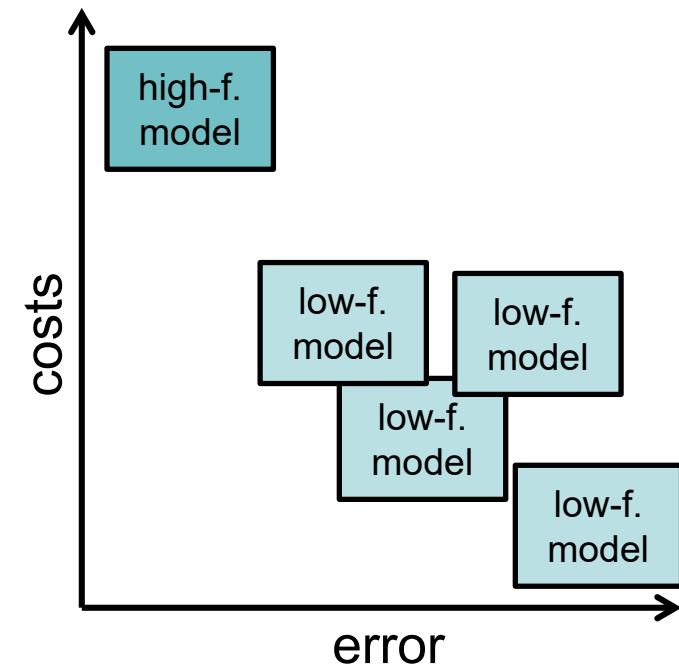
- *Simplified* (e.g. FEM)
  - Coarse mesh
  - Without nonlinearities
  - Less details
- *Projection based*
  - POD
- *Data-fit*
  - Neural networks
  - Spline-interpolation
  - Response surface

# How to combine different models

## Model management strategies:

- **Adaption**
  - Correction of low-f. model with high-f. model
- **Filtering**
  - Low-fidelity filter
  - E.g. importance sampling
- **Fusion**
  - Fusion of low-f. and high-f. output
  - E.g. **MFMC**: control variate method:
    - Reduce the variance of the MFMC-estimator  $\hat{s}$  using control variates  $\bar{y}^i$  with known correlation  $\rho_{1,i}$  to  $\bar{y}^1$ .

$$\hat{s} = \bar{y}_{m_1}^1 + \sum_{i=2}^k \alpha_i \left( \bar{y}_{m_i}^i - \bar{y}_{m_{i-1}}^i \right)$$



# MFMC-Framework example from Peherstorfer

Optimally manage model calls based on their **costs** and **errors**

Get framework on <https://github.com/pehersto/mfmc>

## Pre-Analysis

- Evaluate each model with same samples (low number)
- Store **calculation costs** & **correlation** between models
- Sort models

- Models  $f_i$
- Costs  $w_i$
- Correlation coefficient  $\rho_{1,i} = \frac{\text{Cov}[f_1, f_i]}{\sqrt{\text{Var}[f_1] \text{Var}[f_i]}}$
- Optimal number of realizations  $m_i$

$$\frac{w_{i-1}}{w_i} > \frac{\rho_{1,i-1}^2 - \rho_{1,i}^2}{\rho_{1,i}^2 - \rho_{1,i+1}^2} \longrightarrow \begin{matrix} 0 \leq m_1 \leq \dots \leq m_k \\ |\rho_{1,1}| > \dots > |\rho_{1,k}| \end{matrix}$$

Several runs  $N_{\text{runs}}$   
needed to get  
reliable MSE

## MFMC

- For each calc.-budget:
  - Find **optimal number of realizations** of each model & MFMC-coefficients
  - Run **MFMC-calculations** & calculate **MFMC-estimator**

- Models  $f_i$
- Costs  $w_i$
- Correlation coefficient  $\rho_{1,i}$

For a given calc.- budget  $p$ :

- **Optimal number of realizations**  $m_i = \frac{p \cdot r_i}{r \cdot w}$  with  $r_i = \sqrt{\frac{w_1(\rho_{1,i}^2 - \rho_{1,i+1}^2)}{w_i(1 - \rho_{1,2}^2)}}$

- **MFMC-coefficients**  $\alpha_i = \frac{\rho_{1,i} \sigma_1}{\sigma_i}$

- **MFMC-estimator**  $\hat{\mu}_{\text{mf}} = \bar{y}_{m_1}^1 + \sum_{i=2}^k \alpha_i \left( \bar{y}_{m_i}^i - \bar{y}_{m_{i-1}}^i \right)$
- $\uparrow$  estimate using  $m_1$  evaluations of truth model     
  $\uparrow$  estimate using  $m_i$  evaluations of model i     
  $\uparrow$  estimate using  $m_{i-1}$  evaluations of model i

**MSE &  
compare to  
MC**

- Calculate reference value (with  $f_1$ )
- For each calc.-budget:
  - Run **MC-Simulation** with  $f_1$
- Compare convergence of MSE (**MFMC** vs. **MC**)

Several runs  $N_{\text{runs}}$   
needed to get  
reliable MSE

$$\text{MSE}(\hat{s}) = \frac{1}{N_{\text{runs}}} \sum_{i=1}^{N_{\text{runs}}} \frac{(\hat{s}_i - \hat{s}^{\text{Ref}})^2}{(\hat{s}^{\text{Ref}})^2}$$

**MSE &  
compare to  
MC**

- Calculate reference value (with  $f_1$ )
- For each calc.-budget:
  - Run **MC-Simulation** with  $f_1$
- Compare convergence of MSE (**MFMC** vs. **MC**)

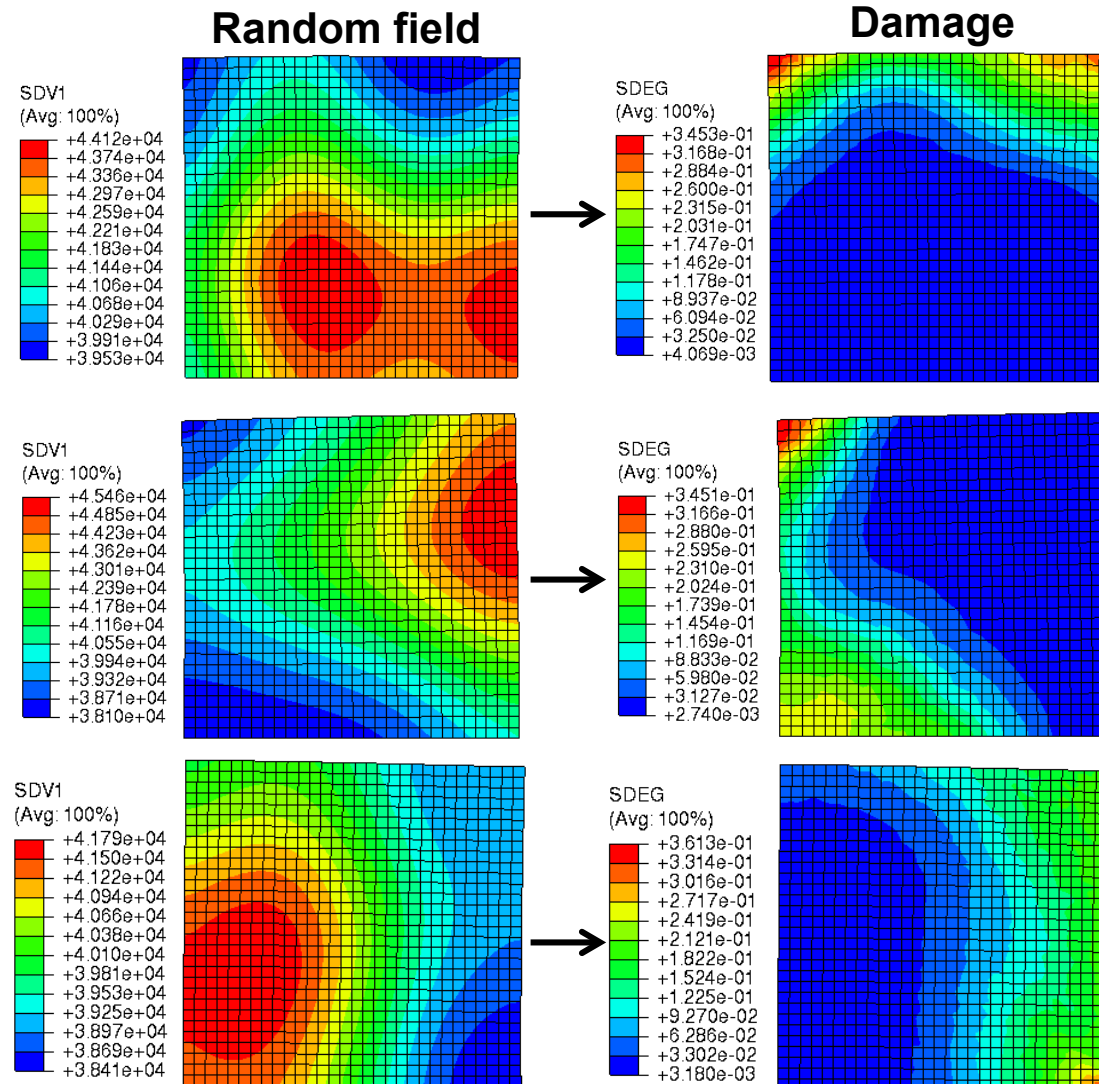
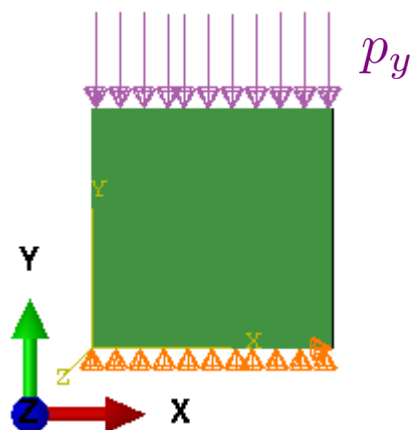
Several runs  $N_{\text{runs}}$   
needed to get  
reliable MSE

$$\text{MSE}(\hat{s}) = \frac{1}{N_{\text{runs}}} \sum_{i=1}^{N_{\text{runs}}} \frac{(\hat{s}_i - \hat{s}^{\text{Ref}})^2}{(\hat{s}^{\text{Ref}})^2}$$

# Example: Damage calculation with random field

## High-fidelity model $f_1$ :

- **2D FEM-Model**
- **Input:** 8 KLE-Coefficients  
(Random field for E)
  - $\mu = 40000 \text{ MPa}$
  - $\sigma = 2000 \text{ MPa}$
  - $l_c = 7.5 \text{ cm}$
- **Output:**  $Y = p_{y,\text{failure}}$
- **Costs:**  $w_1 = 246.36 \text{ s}$



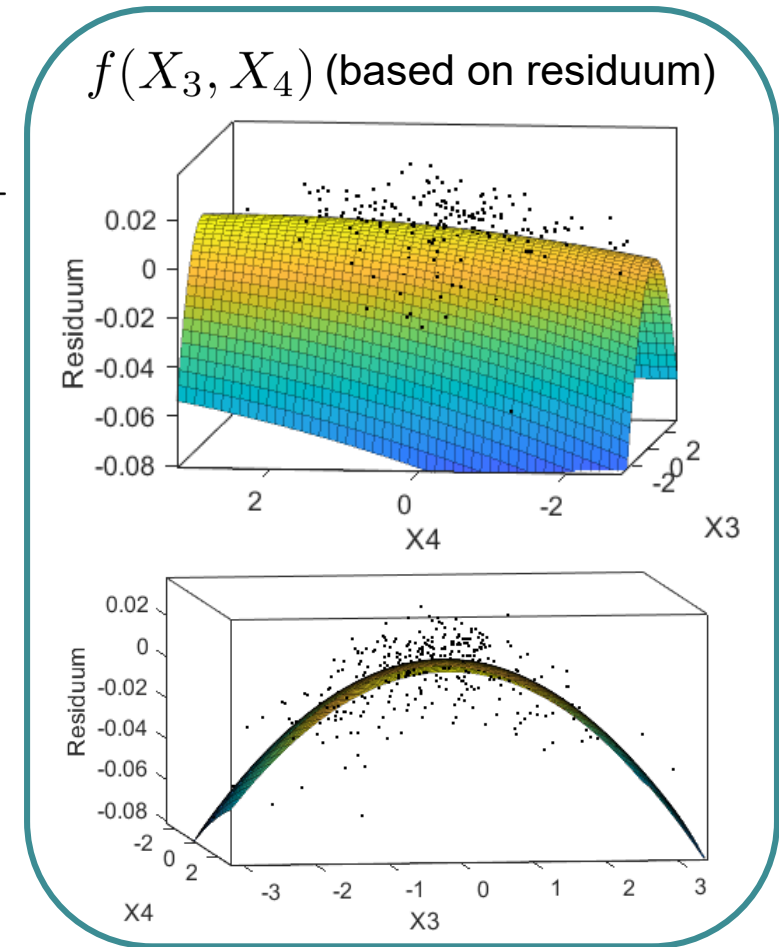
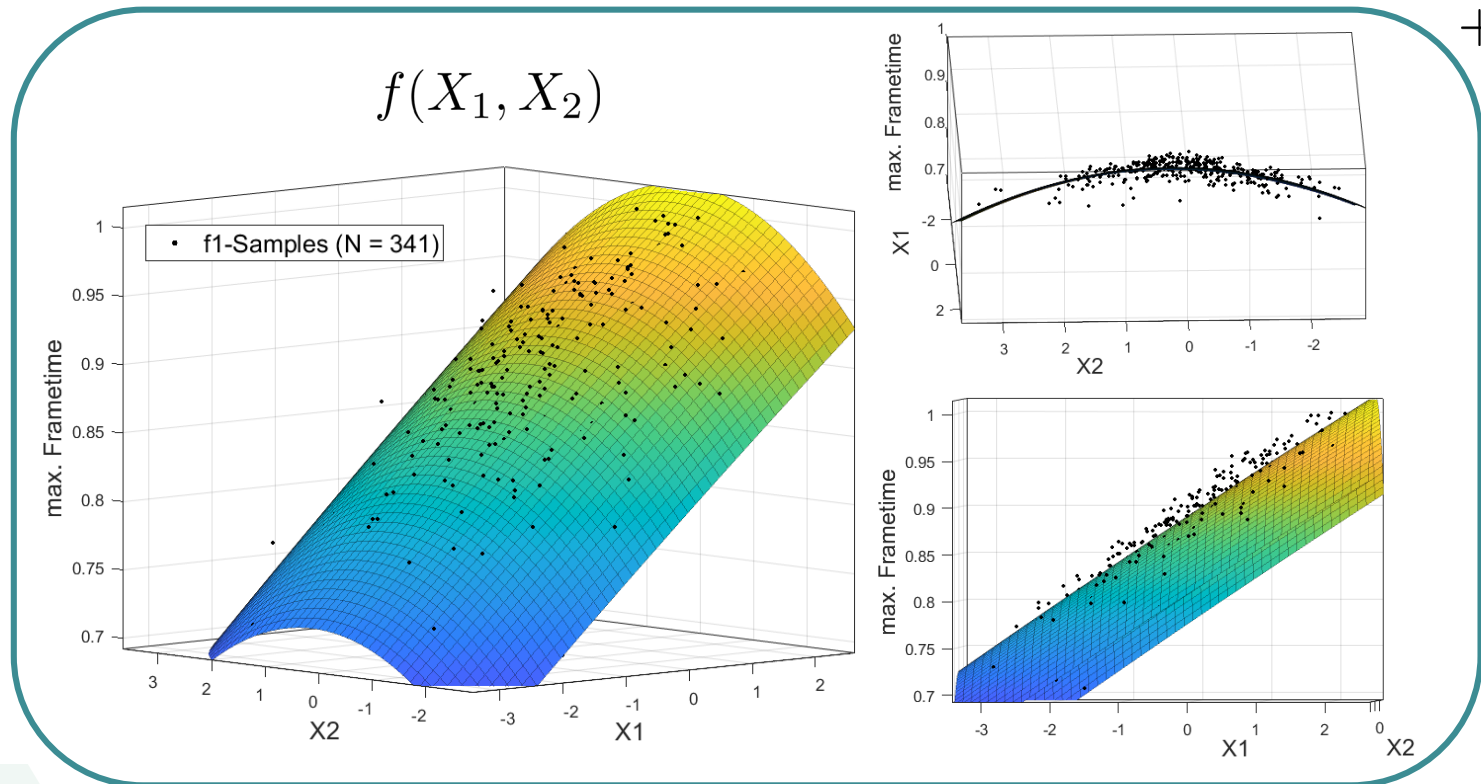
$$p_{y,\text{failure}} = 110.63 \text{ MPa}$$

$$p_{y,\text{failure}} = 106.84 \text{ MPa}$$

$$p_{y,\text{failure}} = 106.01 \text{ MPa}$$

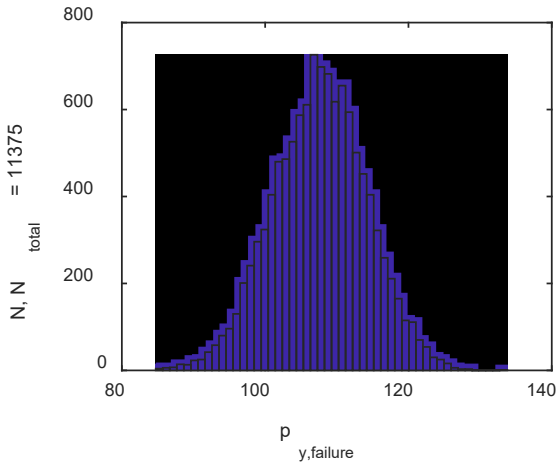
## Low-fidelity model $f_2$ :

- **Data-fit Model (Polynomial surface interpolation)**  $f_2 = f(X_1, X_2) + f(X_3, X_4)$   
(341  $f_1$  calculations used, **degree 2**)
- **Costs:**  $w_3 = 5.03 \cdot 10^{-6}$  s
- **Correlation coefficient:**  $\rho_{1,2} = 0.9595$



MFMC calculation:

|         | MFMC       |            | MC         |                  | Runs for MSE |
|---------|------------|------------|------------|------------------|--------------|
| Budget  | $m_1(f_1)$ | $m_2(f_2)$ | $m_1(f_1)$ | $m_2(f_2)$       |              |
| 500 s   | 2          | 14294      | 3          | $8.5 \cdot 10^6$ | 25           |
| 1000 s  | 4          | 28589      | 5          | $1.7 \cdot 10^7$ | 25           |
| 5000 s  | 20         | 142946     | 21         | $8.5 \cdot 10^7$ | 25           |
| 10000 s | 40         | 285893     | 41         | $1.7 \cdot 10^8$ | 25           |
| 20000 s | 81         | 571787     | 82         | $3.4 \cdot 10^8$ | 25           |



Convergence of mean and variance:

